COMPLEMENTO DEL TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Personalización de Sesiones de Entrenamiento en Auscultación mediante Sistemas de Recomendación.



Curso: 2025/2026

Alumno/a:

José Carlos Ruiz Sánchez

Director/es:

Antonio Jesús Fernandez García

A mi familia y amigos, por estar presente y ayudarme cuando lo necesitaba. Cada experiencia compartida, cada palabra y cada gesto han contribuido a formar quien soy a día de hoy. Gracias por ser parte de este viaje, con todos sus desafíos y alegrías.

ÍNDICE GENERAL

Página

Resumen y Abstract **x**

1. Introducción **1**

1.1. Introducción e interés 1

1.2. Motivación del complemento de trabajo de fin de grado 1

1.3. Objetivos 1

1.4. Planificación 3

2. Background y trabajos relacionados **7**

2.1. Materiales y métodos 7

2.2. Background 8

2.2.1. Sistemas de recomendación: por qué existen 8

2.2.2. La información con la que trabajan: señales explícitas e implícitas 9

2.2.3. Algoritmos básicos 9

2.2.4. Filtrado basado en contenido 9

2.2.5. Filtrado colaborativo 10

2.2.5.1. Colaborativo basado en usuarios 10

2.2.5.2. Colaborativo basado en ítems 11

2.2.6. Retos habituales: arranque en frío, dispersión y sesgos 12

2.2.7. Evaluación: utilidad y calidad de listas recomendadas 12

2.3. Trabajos relacionados 12

3. Visión general **15**

4. Especificación técnica y diseño **19**

4.1. Definición de algoritmos 19

4.1.1. Aleatorio simple 19

4.1.2. Preguntas que no has hecho 19

4.1.3. Categoría concreta 20

4.1.4. Categoría que peor se te da y categoría que mejor se te da 20

4.1.5. Preguntas que tiendes a fallar y preguntas que tiendes a acertar 21
4.1.6. Preguntas que otros usuarios similares fallan y preguntas que otros usuarios similares aciertan 22
4.1.7. Algoritmo basado en ítems: preguntas similares que fallan y preguntas similares que aciertan 23
4.2. Razonamiento de los algoritmos 24
4.2.1. Aleatorio simple 24
4.2.2. Preguntas que no has hecho 25
4.2.3. Categoría concreta 25
4.2.4. Categoría que peor se te da y categoría que mejor se te da 26
4.2.5. Preguntas que tiendes a fallar y preguntas que tiendes a acertar 26
4.2.6. Preguntas que otros usuarios similares fallan y preguntas que otros usuarios similares aciertan 27
4.2.6.1. Cálculo de similitud entre usuarios 28
4.2.7. Algoritmo basado en ítems: preguntas similares que fallan y preguntas similares que aciertan 29
4.2.7.1. Formalización del cálculo de similitud entre preguntas 30
4.2.7.2. Estrategia de selección de preguntas recomendadas 31
4.3. Justificación de las tecnologías utilizadas 31

5. Conclusión y trabajo futuro **33**
5.1. Conclusiones 33
5.2. Trabajo futuro 34
BIBLIOGRAFÍA 34

ÍNDICE DE FIGURAS

1.1. Planificación inicial del complemento. 4
1.2. Planificación real del complemento. 4

2.1. Logotipo de Python. 7
2.2. Logotipo de Flask. 8
2.3. Logotipo de Docker. 8
2.4. Filtrado colaborativo. 10
2.5. Filtrado colaborativo usuarios. 10
2.6. Filtrado colaborativo items. 11

3.1. Diagrama general del CTFG. 16
3.2. Diagrama de arquitectura física del Complemento de Trabajo de Fin de Grado. . 17

ÍNDICE DE TABLAS

1.1. Tiempo dedicado por tarea. 5

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

Este Complemento de Trabajo de fin de grado implementa un sistema de recomendación para una app que entrena a estudiantes de medicina a reconocer sonidos corporales con audios reales. Incluye 11 algoritmos (básicos, basados en rendimiento e interacciones colaborativas por usuarios e ítems) para generar sesiones de práctica personalizadas según historial de aciertos/errores, dominio por categorías y patrones de similitud.

This Bachelor's thesis add-on implements a recommendation system for an app that trains medical students to recognize body sounds from real audio recordings. It integrates 11 algorithms (basic, performance-based, and collaborative user-/item-based) to build personalized practice sessions using past correctness, category mastery, and similarity patterns across users and questions.

1 Introducción

1.1 Introducción e interés

Este Complemento de Trabajo de fin de grado se centra en el diseño de un sistema de recomendación que selecciona automáticamente qué preguntas proponer al estudiante en cada sesión. En una aplicación con un banco de preguntas amplio y en crecimiento, una secuencia fija o una selección manual no escala y desaprovecha información útil generada durante el uso (aciertos, fallos, repeticiones y categorías trabajadas).

El interés del complemento es la implementación de **11 algoritmos** con diferentes requisitos de datos y comportamientos: métodos base (aleatorio, preguntas no realizadas, por categoría), métodos orientados al rendimiento individual (mejor/peor categoría, priorización por tendencia a fallar/acertar) y filtrado colaborativo **basado en usuarios** e **ítems**.

1.2 Motivación del complemento de trabajo de fin de grado

La motivación de este Complemento de Trabajo de Fin de Grado es sustituir la selección de preguntas por un enfoque algorítmico, **objetivo y evaluable**, capaz de adaptarse al comportamiento del usuario y al crecimiento del repositorio de preguntas. Los sistemas de recomendación permiten explotar de forma sistemática las interacciones históricas para generar selecciones más relevantes que un itinerario fijo.

Se ha optado por implementar **múltiples algoritmos** (en lugar de uno único) por dos motivos principales: (1) **robustez ante falta de datos** (problema de *cold-start*), donde los métodos simples funcionan cuando aún no hay suficiente historial, y (2) **comparación y control del criterio**, pudiendo alternar entre enfoques de refuerzo (errores), consolidación (aciertos) o similitud entre usuarios/preguntas. En particular, el filtrado colaborativo y los métodos basados en ítems permiten recomendar a partir de patrones compartidos y relaciones entre preguntas observadas en el uso real.

1.3 Objetivos

El objetivo principal de este complemento de Trabajo de Fin de Grado es diseñar e implementar un sistema de recomendación capaz de seleccionar automáticamente qué preguntas proponer al estudiante en cada sesión, utilizando distintos criterios de recomendación en función del histórico de uso y del contexto de la aplicación.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Implementar un conjunto de algoritmos de recomendación con **criterios heterogéneos**, de forma que el sistema pueda adaptarse a diferentes necesidades del estudiante (refuerzo de errores, consolidación de aciertos, cobertura de contenido o exploración).
- Incorporar métodos que funcionen con **escasez de datos** (situaciones de *cold-start*) mediante estrategias simples y basadas en reglas, y métodos más avanzados cuando exista suficiente información histórica.
- Incluir algoritmos orientados al **rendimiento individual** (por categorías y por preguntas) que prioricen aquello que el usuario tiende a fallar o acertar.
- Integrar técnicas de **filtrado colaborativo** basadas en similitud entre usuarios, para recomendar preguntas a partir de patrones compartidos por estudiantes con comportamientos parecidos.
- Integrar técnicas de recomendación **basadas en ítems** (similitud entre preguntas), utilizando co-ocurrencias observadas en el uso real para sugerir preguntas relacionadas con errores o aciertos.
- Dejar definidos y almacenados los criterios de cada algoritmo en la base de datos para facilitar su **selección, mantenimiento y extensión** conforme crezca el banco de preguntas y el número de usuarios.

En el desarrollo del complemento se han implementado **11 algoritmos** de recomendación. A continuación se enumeran según el criterio aplicado:

- **Aleatorio simple**: elige preguntas al azar sin utilizar información personal ni de otros usuarios.
- **Preguntas que no has hecho**: prioriza preguntas no realizadas; si no existen, selecciona las menos recientes.
- **Categoría concreta**: selecciona preguntas de una categoría elegida.
- **Categoría que peor se te da**: prioriza preguntas de la categoría con peor rendimiento histórico del usuario.
- **Categoría que mejor se te da**: prioriza preguntas de la categoría con mejor rendimiento histórico del usuario.
- **Preguntas que tiendes a fallar**: recomienda preguntas falladas previamente, priorizando aquellas con mayor tasa de error.
- **Preguntas que tiendes a acertar**: recomienda preguntas acertadas previamente, priorizando aquellas con mayor tasa de acierto.
- **Preguntas que otros usuarios similares fallan**: recomienda preguntas falladas por usuarios similares al actual.

- **Preguntas que otros usuarios similares aciertan**: recomienda preguntas acertadas por usuarios similares al actual.
- **Algoritmo basado en ítems – Preguntas similares que fallan**: recomienda preguntas similares a otras que suelen fallarse conjuntamente por distintos usuarios.
- **Algoritmo basado en ítems – Preguntas similares que aciertan**: recomienda preguntas similares a otras que suelen acertarse conjuntamente por distintos usuarios.

En apartados posteriores se describe con mayor detalle el funcionamiento interno de cada algoritmo, los datos necesarios para su ejecución y el criterio exacto de priorización utilizado.

1.4 PLANIFICACIÓN

En este apartado se muestra de forma detallada la planificación seguida en la elaboración del presente Complemento de Trabajo de Fin de Grado. Dado que se trata de un desarrollo acotado y de menor alcance que el Trabajo de Fin de Grado principal, la planificación se organiza en fases más compactas, centradas en la implementación de los algoritmos de recomendación, su integración en la aplicación y la documentación asociada.

De acuerdo con la planificación inicial, el desarrollo del complemento se extendía desde el **29/12/2024** hasta el **31/05/2025**. Dentro de este intervalo se definieron varias fases principales, cada una de ellas con un periodo teórico de ejecución asignado:

- **Planificación inicial**: del **29/12/2024** al **05/01/2025**. Esta fase se centró en la definición del alcance del complemento, la identificación de tareas y la estimación de tiempos.
- **Preparación del anteproyecto**: del **06/01/2025** al **10/01/2025**. Durante este intervalo se concretaron los objetivos del complemento, los criterios de recomendación y la planificación preliminar.
- **Formación con la tecnología utilizada**: del **13/01/2025** al **10/02/2025**. Fase destinada a la adquisición y consolidación de conocimientos sobre las herramientas necesarias para la implementación y sobre los distintos algoritmos a implementar.
- **Desarrollo de los algoritmos**: del **11/02/2025** al **29/04/2025**. Corresponde a la implementación de los distintos criterios de recomendación y su validación básica.
- **Redacción del documento**: del **11/02/2025** al **29/04/2025**. La redacción se planifica en paralelo al desarrollo, con el fin de documentar el proceso de forma progresiva.

A continuación se presentan dos diagramas de Gantt. En la figura 1.1 se recoge la planificación inicial del complemento, basada en los intervalos teóricos asignados a cada fase. En la figura 1.2 se muestra la planificación real, donde se reflejan las fechas efectivas de ejecución y los reajustes temporales realizados durante el desarrollo.

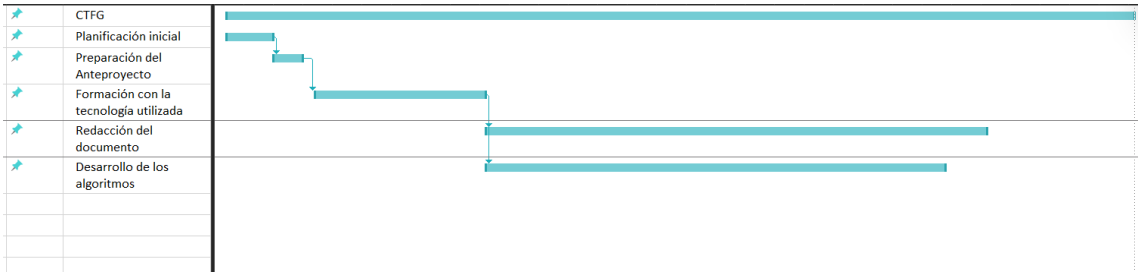


Figura 1.1: Planificación inicial del complemento.

Si se compara la figura 1.1 con la figura 1.2, se observa una desviación respecto al inicio previsto del complemento. Esta divergencia principal se debe a la hospitalización del redactor de este documento y a la imposibilidad de trabajar en el proyecto durante aproximadamente un mes. Como consecuencia, fue necesario reajustar la planificación para redistribuir las tareas y poder abordar el complemento dentro de los plazos establecidos, tal y como se aprecia en la figura 1.2.

En la planificación real, el complemento abarca un total de **287 días**, desde el **29/12/2024** hasta el **31/01/2026**.La fase de **planificación inicial** se desarrolla entre el **29/12/2024** y el **05/01/2025**, seguida de la **preparación del anteproyecto**, que ocupa del **06/01/2025** al **10/01/2025**. La **formación con los algoritmos necesarios** se realiza del **01/03/2025** al **22/03/2025**. La **formación con la tecnología utilizada** se realiza del **15/03/2025** al **09/04/2025**. A continuación, el **diseño conceptual de los algoritmos** se concentra entre el **24/03/2025** y el **31/03/2025**, mientras que la **definición detallada de los algoritmos** se realiza del **10/06/2025** al **30/06/2025**. Posteriormente la **implementación de los algoritmos** se realiza del **5/07/2025** al **15/09/2025**. Por último, la **redacción del documento** se prolonga en paralelo desde el **02/02/2025** hasta el **20/12/2025**. Esta reorganización temporal refleja el reajuste necesario para absorber los retrasos iniciales, manteniendo el foco en la implementación y documentación del sistema de recomendación.

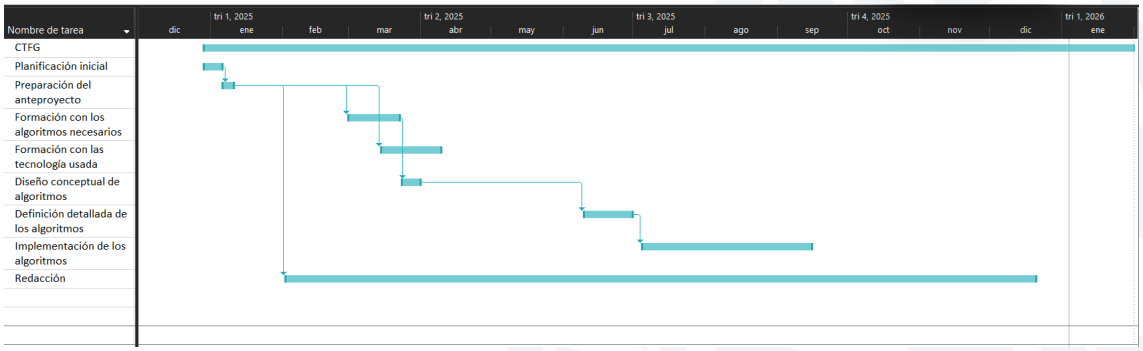


Figura 1.2: Planificación real del complemento.

Para concluir, y con el objetivo de verificar el cumplimiento de las horas estimadas en el complemento, se presenta la tabla 1.1 que detalla la distribución de horas por tarea y expone el tiempo dedicado a cada fase del desarrollo. Con el fin de asegurar la precisión en la medición del tiempo, se ha empleado la técnica de Pomodoro, un método de gestión del tiempo que divide el trabajo en intervalos de 25 minutos separados por breves descansos.

En la tabla 1.1 se puede observar el tiempo real dedicado a cada una de las tareas registradas. En concreto, para la tarea de **planificación** se han dedicado **2 horas**, mientras que la **preparación del anteproyecto** ha requerido **5 horas**. La **formación con los algoritmos necesarios** ha supuesto **14 horas**, y la **formación con la tecnología utilizada** ha requerido **6 horas**, siendo ambas necesarias para adquirir los conocimientos previos al desarrollo del complemento.

En relación con el diseño de la solución propuesta, la **definición conceptual de los algoritmos** ha requerido **5 horas**, mientras que la **definición detallada de los algoritmos** ha supuesto **12 horas** adicionales, destinadas a concretar su funcionamiento y estructura. La **implementación de los algoritmos** ha concentrado la mayor carga de trabajo, con un total de **25 horas**, debido a la complejidad asociada a su desarrollo y validación.

Por último, la **redacción del documento** ha requerido **37,30 horas**, destinadas a describir el trabajo realizado, justificar las decisiones adoptadas y presentar los resultados obtenidos.

Sumando todas las tareas, el complemento acumula un total de **113,3 horas** reales de dedicación. Esta carga de trabajo se considera coherente con el alcance del complemento, al permitir adquirir la formación necesaria, diseñar e implementar los algoritmos propuestos y documentar el proceso de forma adecuada.

Tarea	**Horas (reales)**
Planificación	2,00
Preparación del anteproyecto	5,00
Formación con los algoritmos necesarios	14,30
Formación con la tecnología utilizada	6,25
Definición conceptual de los algoritmos	5,00
Definición detallada de los algoritmos	12,45
Implementación de los algoritmos	25,50
Redacción del documento	37,30

Tabla 1.1: Tiempo dedicado por tarea.

2 Background y trabajos relacionados

2.1 Materiales y métodos

En esta sección se listan las herramientas, métodos y materiales utilizados que han permitido la elaboración de este Complemento de Trabajo de Fin de Grado.

1. Elaboración del código fuente.

Para el desarrollo del código fuente de la aplicación se han utilizado diversas librerías y herramientas con el objetivo de facilitar la construcción de cada uno de los componentes del sistema.

- **Python**: Python es un lenguaje de programación de alto nivel, interpretado y de propósito general. Es reconocido mundialmente por su sintaxis clara, legibilidad y mayor simplicidad para programar que otros lenguajes. También ofrece una serie de herramientas versátiles para el desarrollo de algoritmos. Fue creado por Guido van Rossum y la primera versión se publicó en 1991.



Figura 2.1: Logotipo de Python.

- **Flask**: Es un framework para Python que sirve para la creación de aplicaciones web y APIs REST. Fue creado por Armin Ronacher. La principal ventaja que te ofrece Flask es que puedes agregar los componentes que te hagan falta para que sea lo más ligero posible.



Figura 2.2: Logotipo de Flask.

4. Despliegue.

- **Docker**: Docker es una aplicación pensada para el despliegue de componentes en contenedores para la aislar el código e instalar las dependencias necesarias para que funcione en cualquier dispositivo. Es especialmente útil gracias a que permite encapsular el código sin que entre en conflicto con otras partes de la aplicación.

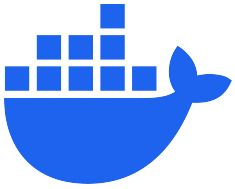


Figura 2.3: Logotipo de Docker.

2.2 Background

Esta sección presenta el contexto conceptual necesario para comprender, desde un punto de vista informático, qué son los sistemas de recomendación y cuáles son las ideas principales detrás de sus algoritmos. El objetivo es explicar por qué existen estos sistemas, qué tipos de enfoques suelen emplearse y qué limitaciones aparecen de forma natural cuando se trabaja con historiales de interacción.

2.2.1 Sistemas de recomendación: por qué existen

En cualquier entorno donde haya un catálogo amplio de opciones, llega un momento en el que elegir manualmente deja de ser práctico y emerge el problema de la sobrecarga de información. Los sistemas de recomendación (RS) son herramientas diseñadas para ayudar a los usuarios a descubrir elementos relevantes y reducir esta sobrecarga priorizando qué mostrar a continuación [1, 2].

Durante años, estos sistemas se han usado para ayudar a filtrar información y reducir la carga de decisión del usuario. Su función real no se limita exclusivamente a predecir gustos: muchas veces su objetivo es sostener una experiencia coherente cuando el catálogo crece,

introduciendo variedad, controlando repeticiones y, en general, tomando decisiones repetibles y justificables donde una secuencia fija sería rígida o ineficiente [1].

2.2.2 La información con la que trabajan: señales explícitas e implícitas

Los recomendadores se alimentan de datos de interacción. En algunos casos, estos datos son valoraciones explícitas (por ejemplo, puntuaciones dadas por los usuarios). Sin embargo, en la mayoría de aplicaciones reales, la interacción registrada es implícita: clics, vistas, compras o tiempo de permanencia, que ofrecen señales indirectas de preferencia [2, 3].

Estas señales implícitas tienen dos particularidades clave. La primera es que la ausencia de interacción es ambigua: no siempre refleja rechazo, sino a menudo falta de exposición o de oportunidad [2]. La segunda es que la fuerza de la evidencia no es uniforme: una interacción esporádica aporta menos información que un patrón consistente a lo largo del tiempo. Por eso, los algoritmos suelen combinar criterios y mecanismos de exploración para no concluir demasiado a partir de pocos datos [2].

2.2.3 Algoritmos básicos

Es importante entender los algoritmos básicos y en qué se basan. Estas estrategias simples no intentan descubrir estructuras complejas en los datos, sino aplicar reglas transparentes y controlables que, en muchos casos, funcionan con poco historial y sirven como referencia base para métodos más sofisticados [1].

Entre estas reglas simples se encuentran la selección aleatoria, que introduce exploración; la popularidad, que prioriza ítems usados con frecuencia; y la novedad, que facilita el descubrimiento de nuevo contenido. Estas estrategias aportan estabilidad, sirven como punto de comparación y permiten gobernar el comportamiento del sistema desde criterios fácilmente interpretables.

2.2.4 Filtrado basado en contenido

El filtrado basado en contenido (content-based filtering) recomienda ítems analizando las características del contenido y el perfil individual del usuario. En este enfoque, cada ítem se describe mediante un conjunto de atributos (por ejemplo, etiquetas, categorías o características extraídas automáticamente), y el perfil del usuario se construye a partir de los ítems con los que ha interactuado previamente [1].

La recomendación se realiza comparando la similitud entre el perfil del usuario y los ítems disponibles, priorizando aquellos cuyo contenido se asemeja a lo que el usuario ha consumido en el pasado. Este enfoque no depende de otros usuarios y permite una personalización directa incluso cuando el sistema tiene pocos usuarios activos.

Entre sus ventajas se encuentra la capacidad de explicar fácilmente por qué se recomienda un ítem, ya que la decisión se basa en atributos conocidos. Sin embargo, este enfoque tiende a reforzar preferencias previas, limitando la diversidad, y depende fuertemente de la calidad de la representación del contenido.

2.2.5 Filtrado colaborativo

El filtrado colaborativo se basa en la idea de usar patrones de comportamiento agregados en la comunidad para recomendar ítems relevantes sin depender de descripciones detalladas del contenido [1]. Este enfoque permite personalización explotando similitudes implícitas entre usuarios o entre ítems, aprendidas directamente de los historiales de interacción. Como se puede ver en la figura 2.4

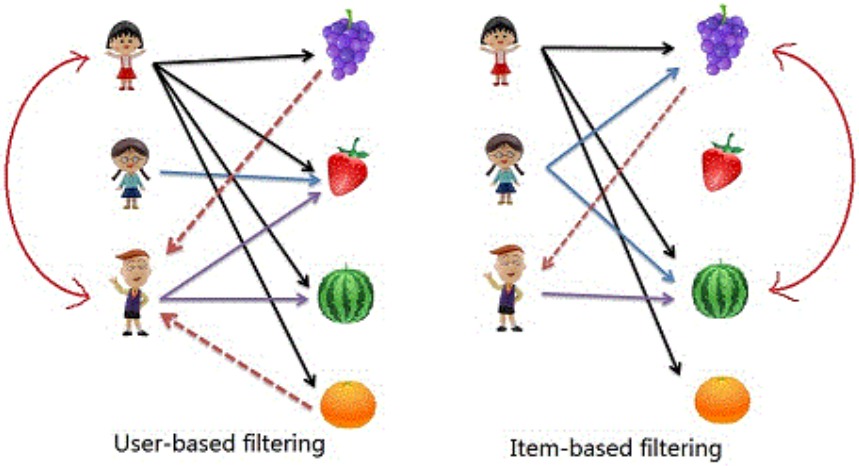


Figura 2.4: Filtrado colaborativo.

Colaborativo basado en usuarios

En el enfoque basado en usuarios, el sistema identifica usuarios con patrones de interacción similares. La suposición central es que, si dos usuarios han coincidido en sus elecciones en el pasado, es probable que sigan haciéndolo en el futuro [1]. A partir de estos vecinos similares, se recomiendan ítems que el usuario objetivo aún no ha consumido.

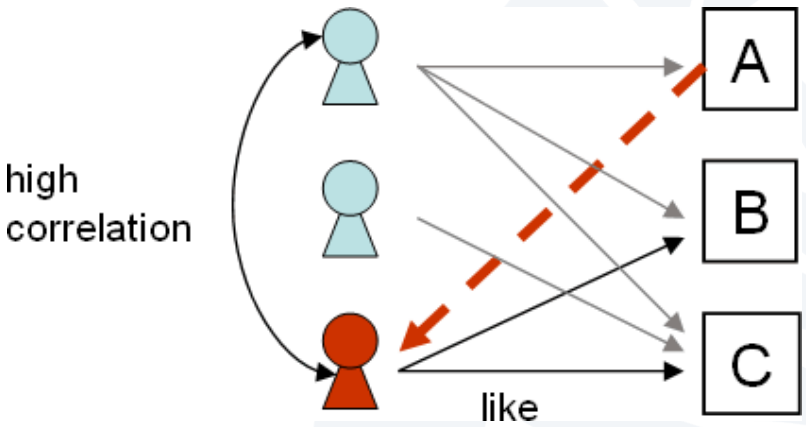


Figura 2.5: Filtrado colaborativo usuarios.

En el filtrado colaborativo basado en usuarios, como se puede ver en la figura 2.5, el razonamiento parte de una idea sencilla: personas que han tomado decisiones similares en

el pasado tienden a tener gustos parecidos. El sistema observa los historiales de interacción y busca otros usuarios cuyo comportamiento se parezca al del usuario objetivo.

Por ejemplo, si dos usuarios han interactuado con muchos de los mismos ítems (han visto, comprado o valorado contenido similar), el sistema los considera usuarios similares. A partir de ahí, se recomiendan al usuario objetivo aquellos ítems que han resultado relevantes para sus usuarios similares, pero que él todavía no ha consumido.

Este método es intuitivo y conceptualmente simple, pero presenta limitaciones cuando los historiales son escasos o muy dispersos. Además, el coste computacional puede crecer significativamente a medida que aumenta el número de usuarios.

Colaborativo basado en ítems

El enfoque basado en ítems invierte la perspectiva: dos ítems se consideran similares si suelen aparecer juntos en los historiales de muchos usuarios. En lugar de buscar usuarios parecidos, el sistema recomienda ítems similares a aquellos con los que el usuario ya ha interactuado [4].

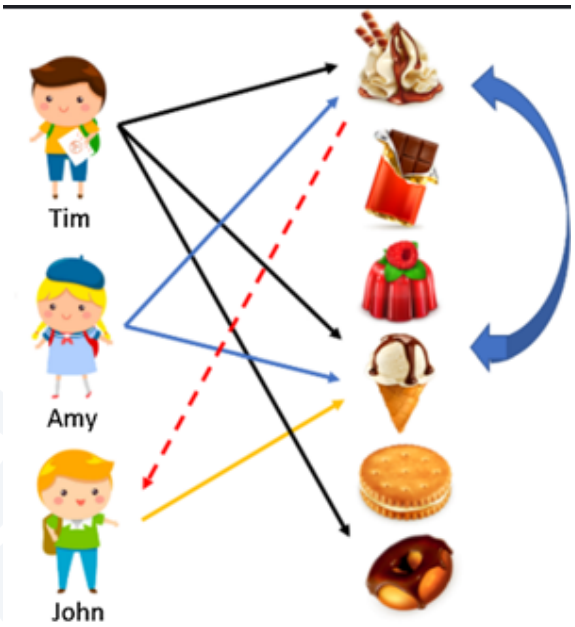


Figura 2.6: Filtrado colaborativo items.

El filtrado colaborativo basado en ítems sigue un razonamiento ligeramente distinto, como se puede ver en la figura 2.5: los ítems que suelen ser consumidos juntos están relacionados entre sí. En lugar de buscar usuarios similares, el sistema analiza cómo se combinan los ítems en los historiales de muchos usuarios.

Por ejemplo, si un gran número de usuarios ha interactuado tanto con el ítem A como con el ítem B, el sistema infiere que ambos ítems son similares, aunque no conozca sus características internas. Cuando un usuario interactúa con un ítem concreto, el sistema recomienda otros ítems que han demostrado estar estrechamente relacionados con él en el comportamiento colectivo.

Estas relaciones entre ítems suelen ser más estables en sistemas a gran escala y permiten reutilizar los cálculos de similitud de forma eficiente. No obstante, este enfoque tiene dificultades para manejar ítems nuevos o con pocas interacciones registradas.

2.2.6 Retos habituales: arranque en frío, dispersión y sesgos

Los recomendadores enfrentan retos comunes. El primero es el arranque en frío: cuando un usuario o ítem es nuevo, no hay historial suficiente para personalizar recomendaciones de forma confiable [1]. El segundo es la dispersión en catálogos grandes, donde cada usuario solo interactúa con una pequeña fracción del contenido. El tercero es el sesgo inducido por la propia recomendación, como el sesgo de popularidad que puede amplificarse con cada iteración del sistema [5, 1].

Por estas razones, en sistemas reales se busca un equilibrio entre explotar patrones conocidos y explorar alternativas que mejoren la diversidad y reduzcan la homogeneización de las recomendaciones.

2.2.7 Evaluación: utilidad y calidad de listas recomendadas

Evaluar un sistema de recomendación va más allá de medir si acierta predicciones. En muchos casos, es más importante entender si las listas recomendadas son útiles para los objetivos del sistema, midiendo aspectos como relevancia percibida, diversidad, novedad y cobertura de catálogo [3, 1]. También se distingue entre evaluación con datos históricos en laboratorio y evaluación en contextos con usuarios reales, que captura el impacto en la experiencia de uso.

2.3 Trabajos relacionados

En esta sección se recogen ejemplos de productos comerciales en los que la selección y ordenación de contenidos para cada persona depende de algoritmos de recomendación, de forma análoga a los criterios de selección que implementa el complemento desarrollado.

Un caso clásico es Amazon, donde las recomendaciones de productos se construyen a partir del comportamiento previo de compra y de la relación observada entre productos dentro del catálogo. Un enfoque muy documentado en este contexto es la recomendación basada en similitud entre productos, en la que se agregan elementos similares a los ya comprados o valorados para sugerir nuevos artículos [6].

En el ámbito audiovisual, Netflix utiliza un sistema de recomendaciones personalizado para presentar en la página de inicio títulos y colecciones ajustadas a los hábitos de visualización de cada perfil y a patrones observados en otros perfiles con comportamiento parecido [7, 8].

En música, Spotify ofrece recomendaciones mediante listas personalizadas como Discover Weekly, que se actualizan periódicamente y ajustan su contenido en función del uso y la interacción del usuario con la plataforma [9].

Por último, TikTok emplea sistemas de recomendación para ordenar y mostrar vídeos en la sección Para ti, utilizando como señales acciones de interacción como seguir cuentas, indicar que gusta un contenido o la forma de consumo de los vídeos [10].

3 Visión general

En este capítulo se presenta una visión global del complemento desarrollado, con el objetivo de contextualizar su funcionamiento dentro de la arquitectura general de la aplicación descrita en el Trabajo de Fin de Grado. La intención de esta sección es ofrecer una imagen clara del papel que desempeña el sistema de recomendación, qué información utiliza y cómo se integra en el flujo general de uso antes de entrar en el detalle técnico de los algoritmos y de su implementación.

El complemento implementado corresponde al sistema de recomendación de la aplicación y actúa como el módulo encargado de generar sesiones de práctica personalizadas para los estudiantes. A partir de una petición realizada por la aplicación principal, el sistema de recomendación analiza el contexto del usuario y selecciona automáticamente un conjunto de preguntas que conforman la sesión. Estas preguntas se devuelven en un formato homogéneo que permite a la aplicación presentar los audios, las opciones de respuesta y la información asociada sin depender de los detalles internos del algoritmo utilizado.

Desde un punto de vista funcional, el sistema de recomendación se sitúa entre la base de datos de la aplicación y la interfaz de usuario. Por un lado, consume información histórica almacenada en la base de datos, como las respuestas previas del usuario, sus aciertos y fallos, las categorías trabajadas y las interacciones generadas por otros usuarios. Por otro lado, produce como salida una lista de preguntas recomendadas que se ajustan a un criterio concreto de selección. Esta separación permite desacoplar la lógica de recomendación del resto de la aplicación, facilitando su mantenimiento y evolución.

En la Figura 3.1 se muestra un esquema simplificado de la arquitectura general de la aplicación, destacando la posición del sistema de recomendación dentro del flujo global. Este diagrama permite visualizar cómo el complemento se integra con el resto de componentes y cómo participa en la generación de las sesiones de práctica.

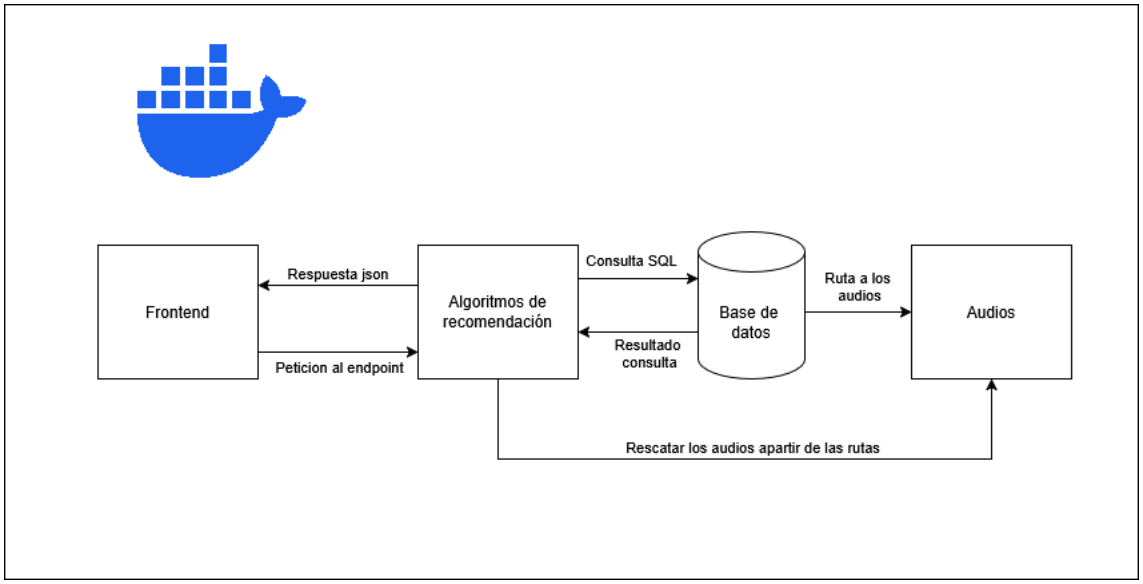


Figura 3.1: Diagrama general del CTFG.

Una de las características principales del complemento es que no se limita a un único método de recomendación. En su lugar, se ha diseñado un conjunto de algoritmos con criterios heterogéneos, que pueden activarse según el contexto de uso o el estado del historial disponible. Esto permite cubrir distintos escenarios, como usuarios nuevos sin historial, usuarios con patrones de error bien definidos o situaciones en las que resulta útil aprovechar información colectiva generada por otros estudiantes.

De manera general, el flujo de funcionamiento del sistema comienza cuando la aplicación solicita una sesión de práctica indicando el algoritmo a utilizar y los parámetros necesarios (número de preguntas, categoría si procede, identificador del usuario, etc.). A continuación, el algoritmo seleccionado construye un conjunto de preguntas candidatas en función de su criterio específico. Sobre ese conjunto se aplica un proceso de selección que da lugar a las preguntas finales de la sesión, garantizando que se cumplen las restricciones necesarias para la generación de preguntas tipo test, como la existencia de una única respuesta correcta y varias alternativas incorrectas válidas.

El resultado del proceso es una sesión completa y lista para su uso, que incluye no solo las preguntas seleccionadas, sino también la información necesaria para su presentación y evaluación. Este enfoque permite que todos los algoritmos compartan una estructura de salida común, a pesar de basarse en criterios internos muy distintos.

En la figura 3.2 se muestra la arquitectura física del complemento. El Frontend, desarrollado en React, actúa como cliente y se comunica con el microservicio de algoritmos de recomendación a través de peticiones HTTP. Cada algoritmo se despliega de manera independiente dentro de un contenedor Docker, manteniendo así el principio de aislamiento propio de una arquitectura basada en microservicios.

Cada algoritmo es independiente entre sí, y se conectan a una api de algoritmos. Este enfoque permite escalabilidad en el sentido de poder añadir algoritmos en el futuro o eliminarlos

conforme la situación lo requiera. También al ser cada algoritmo un archivo independiente, permite que se puedan modificar sin afectar el resto de algoritmos de recomendación.

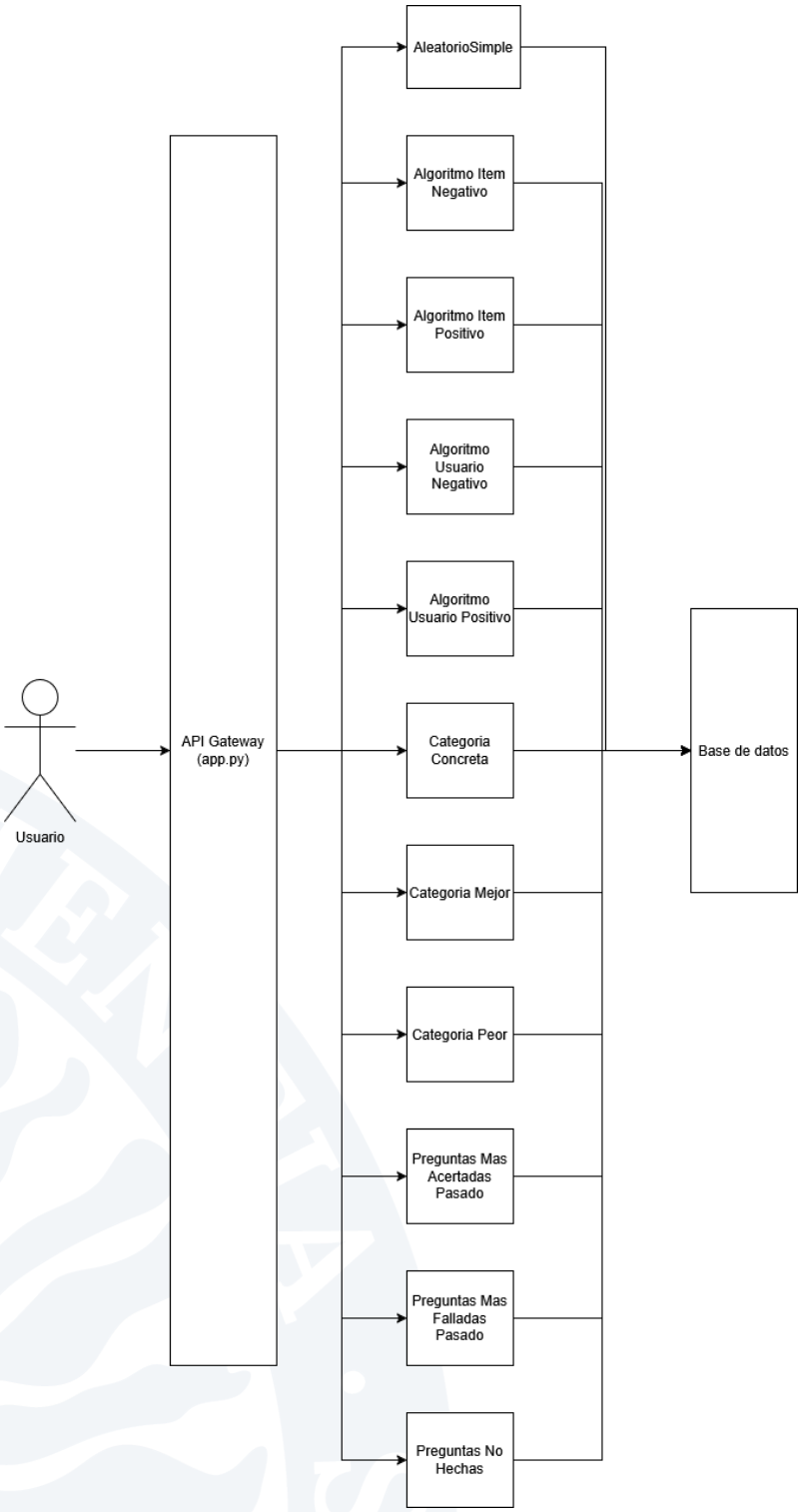


Figura 3.2: Diagrama de arquitectura física del Complemento de Trabajo de Fin de Grado.

En resumen, este capítulo proporciona una visión de alto nivel del complemento de recomendación desarrollado en el marco del Trabajo de Fin de Grado, destacando su integración en la aplicación, su carácter modular y su capacidad para adaptarse a distintos perfiles de usuario. En los capítulos siguientes se detallará con mayor profundidad la especificación técnica de los algoritmos y el razonamiento que justifica cada uno de los criterios de recomendación implementados.

4 Especificación técnica y diseño

4.1 Definición de algoritmos

4.1.1 Aleatorio simple

El algoritmo **aleatorio simple** es la forma más directa de recomendación: en lugar de intentar inferir qué es más adecuado para un usuario, selecciona preguntas de la base de datos **al azar**, sin utilizar información personal, historial previo ni datos agregados de otros usuarios. En términos conceptuales, su función es puramente operativa: ante un conjunto de preguntas disponibles, produce una propuesta de preguntas candidatas mediante una elección no guiada por preferencias.

Este tipo de algoritmo se utiliza con mucha frecuencia como **método de referencia** para evaluar sistemas de recomendación. La razón es sencilla: si un método más avanzado no supera de forma clara a la selección aleatoria, entonces probablemente no está aprovechando la información disponible o el problema está mal formulado. Por ello, la recomendación aleatoria suele aparecer en la literatura y en la práctica como punto de comparación para medir el valor real de técnicas más complejas.

Otra característica importante del enfoque aleatorio es su utilidad en fases de **arranque en frío**. Cuando un usuario es nuevo o el sistema todavía no ha acumulado suficientes interacciones, cualquier intento de personalización puede ser frágil o directamente inviable. En esos casos, un método aleatorio garantiza que el sistema puede seguir funcionando y, al mismo tiempo, contribuye a generar las primeras interacciones que más adelante permitirán aplicar técnicas personalizadas con mayor fiabilidad.

Además, el componente aleatorio se relaciona con un objetivo habitual en recomendación: la **exploración**. Si un sistema solo propone lo que ya conoce o lo que históricamente ha funcionado, corre el riesgo de volverse repetitivo y de dejar sin oportunidades a preguntas menos expuestas. Introducir aleatoriedad, ya sea como algoritmo completo o como parte de una estrategia mixta, es una manera de mantener variedad y de evitar bucles en los que el sistema refuerza siempre las mismas decisiones.

4.1.2 Preguntas que no has hecho

El algoritmo **preguntas que no has hecho** pertenece a la familia de métodos básicos guiados por reglas. Su idea es sencilla: en lugar de intentar estimar qué preguntas se adaptan mejor al usuario a partir de patrones complejos, prioriza el objetivo de **cobertura del**

contenido. Dicho de otra forma, intenta que el usuario avance por el banco de preguntas y no se quede practicando siempre un subconjunto reducido.

El criterio principal consiste en seleccionar, de manera aleatoria, preguntas que el usuario todavía no ha realizado. Esta decisión tiene dos consecuencias positivas. En primer lugar, mantiene una progresión natural por el catálogo, lo cual es útil cuando el banco de preguntas crece o cuando el usuario inicia su práctica y todavía no existe un historial rico. En segundo lugar, introduce variedad de forma controlada: hay aleatoriedad, pero está restringida al conjunto de preguntas nuevas para el usuario, evitando repeticiones innecesarias.

Cuando el usuario ya ha realizado todas las preguntas disponibles, el algoritmo cambia a un criterio de revisión: propone aquellas que **más tiempo llevan sin hacerse**. Conceptualmente, esto convierte el método en una estrategia mixta entre descubrimiento y repaso. Una vez agotadas las preguntas nuevas, la selección por antigüedad favorece que vuelvan a aparecer preguntas que quedaron atrás, ayudando a refrescar contenidos y reduciendo el riesgo de que lo aprendido se degrade por falta de exposición.

En conjunto, se trata de un algoritmo muy útil como base porque funciona con poca información, es fácil de justificar y mantiene dos objetivos prácticos: primero, asegurar que el usuario recorra el catálogo; y segundo, cuando ya no queda contenido nuevo, sostener el repaso dando prioridad a lo menos reciente.

4.1.3 Categoría concreta

El algoritmo **categoría concreta** es un método basado en reglas cuyo objetivo es restringir la recomendación a un subconjunto bien definido del banco de preguntas. En lugar de intentar personalizar la selección a partir de patrones de acierto o similitudes con otros usuarios, este enfoque parte de una idea más simple: si el contenido está organizado por categorías, se puede elegir deliberadamente practicar dentro de una de ellas.

Desde un punto de vista conceptual, este algoritmo funciona como un filtro previo. Primero delimita el espacio de búsqueda, quedándose únicamente con las preguntas que pertenecen a la categoría seleccionada, y después realiza una selección aleatoria dentro de ese conjunto. Esta combinación aporta dos propiedades útiles. Por un lado, ofrece control sobre el contenido, ya que permite focalizar la práctica en un tema concreto sin dispersarse por el resto del catálogo. Por otro lado, mantiene variedad dentro de la categoría mediante la aleatoriedad, evitando que la sesión se convierta en un recorrido completamente predecible.

Este tipo de recomendación se utiliza habitualmente cuando se quiere reforzar un bloque temático o cuando el usuario necesita una práctica dirigida. También puede servir como mecanismo de organización en sistemas con catálogos grandes, ya que reduce la complejidad de la elección al acotar la recomendación a un ámbito específico. Su principal limitación es que, al depender de una categoría predefinida, no adapta automáticamente la selección al rendimiento del usuario ni aprovecha patrones colectivos, sino que prioriza la claridad del criterio y el control sobre el contenido recomendado.

4.1.4 Categoría que peor se te da y categoría que mejor se te da

Los algoritmos **categoría que peor se te da** y **categoría que mejor se te da** forman parte de los métodos de recomendación basados en el rendimiento. A diferencia de los enfoques

puramente aleatorios o de los filtros por categoría elegida manualmente, aquí la categoría no se fija de antemano, sino que se determina automáticamente a partir del comportamiento previo del propio usuario. La idea general es utilizar el historial como una señal para decidir en qué bloque de contenido tiene más sentido centrar la siguiente sesión.

Ambos algoritmos se apoyan en el mismo principio: el sistema estima el desempeño del usuario en cada categoría a partir de sus intentos previos y, a partir de esa información, selecciona una categoría objetivo. En el caso de la **categoría que peor se te da**, se busca orientar la sesión hacia el refuerzo, priorizando el bloque de contenido donde el usuario muestra más dificultades. En cambio, la **categoría que mejor se te da** persigue un objetivo complementario: consolidar lo ya dominado, reforzando la estabilidad del conocimiento mediante práctica en aquello que el usuario tiende a resolver con mayor éxito.

Una vez identificada la categoría objetivo, la recomendación se restringe a preguntas pertenecientes a ese conjunto y se eligen preguntas dentro de él. Conceptualmente, esto combina dos ideas simples. Por un lado, el rendimiento actúa como criterio de selección de la categoría, lo que introduce personalización sin necesidad de comparar con otros usuarios. Por otro lado, la elección de preguntas dentro de la categoría permite mantener variedad, evitando que la práctica sea siempre exactamente la misma, aunque el foco temático esté claramente definido.

Este tipo de algoritmos resulta útil porque convierte el historial del usuario en una decisión directa y fácil de interpretar. Al mismo tiempo, comparte una limitación habitual de los métodos basados en rendimiento: para que la estimación del desempeño por categoría sea mínimamente representativa, es necesario que exista un número suficiente de interacciones previas en cada categoría. Cuando el historial aún es escaso, el sistema no puede distinguir con fiabilidad qué categoría es la mejor o la peor, por lo que suele ser necesario apoyarse en estrategias más generales hasta que se acumule información.

4.1.5 Preguntas que tiendes a fallar y preguntas que tiendes a acertar

Los algoritmos **preguntas que tiendes a fallar** y **preguntas que tiendes a acertar** son dos variantes de un mismo enfoque: la recomendación basada en el rendimiento del usuario a nivel de pregunta. En lugar de centrar la decisión en categorías completas, estos métodos se fijan en el historial de cada pregunta y en cómo le ha ido al usuario con ella a lo largo del tiempo. La idea es convertir el registro de aciertos y fallos en un criterio directo de selección, de forma que la sesión se adapte a lo que el usuario ha demostrado dominar o, por el contrario, a aquello en lo que suele equivocarse.

En el caso de **preguntas que tiendes a fallar**, la recomendación se orienta al refuerzo. El algoritmo prioriza preguntas que el usuario ya ha fallado previamente y da más importancia a aquellas en las que los errores son más frecuentes o más persistentes. Conceptualmente, esto persigue un objetivo muy claro: concentrar la práctica en los puntos débiles que el propio historial revela, de modo que la repetición tenga intención y no sea solo repetición por azar. Este enfoque resulta especialmente útil cuando se quiere corregir fallos recurrentes, porque evita dispersar la sesión en preguntas donde el usuario ya muestra soltura.

La variante **preguntas que tiendes a acertar** aplica la misma lógica, pero con un objetivo distinto. En este caso se priorizan preguntas que el usuario ha resuelto correctamente en el

pasado, dando más peso a aquellas en las que el acierto es más consistente. La finalidad aquí no es atacar un punto débil, sino consolidar lo ya aprendido y mantener estable el rendimiento. Además, este tipo de recomendación puede tener un efecto positivo sobre la experiencia de uso, ya que tiende a sostener la sensación de progreso y reducir frustración en sesiones donde un exceso de preguntas difíciles podría desmotivar.

Ambos enfoques comparten un requisito y una limitación natural. Necesitan que exista historial suficiente por pregunta para que la estimación sea representativa; con pocas interacciones, es fácil confundir un fallo puntual con una dificultad real o interpretar un acierto aislado como dominio. Y, si se aplican de manera estricta, pueden reducir la exposición a preguntas nuevas: el algoritmo de fallos tiende a insistir en lo problemático y el de aciertos tiende a repetir lo cómodo. Por eso, en términos conceptuales, suelen entenderse como criterios muy útiles para sesiones con una intención clara, pero que funcionan mejor cuando se equilibran con estrategias que aporten variedad y cobertura del banco de preguntas.

4.1.6 Preguntas que otros usuarios similares fallan y preguntas que otros usuarios similares aciertan

Los algoritmos **preguntas que otros usuarios similares fallan** y **preguntas que otros usuarios similares aciertan** pertenecen a la familia del **filtrado colaborativo basado en usuarios**. La idea de fondo es que, cuando varias personas responden a un conjunto de preguntas dentro de la misma aplicación, se generan patrones de comportamiento. Si dos usuarios tienden a acertar y fallar en preguntas parecidas, es razonable tratarlos como usuarios similares, y utilizar lo que ocurre en uno para guiar recomendaciones en el otro.

El enfoque comienza construyendo un perfil del usuario a partir de su historial de respuestas. Después, el sistema busca dentro de la comunidad otros usuarios que hayan respondido a un conjunto comparable de preguntas y cuyo patrón de aciertos y fallos se parezca al del usuario objetivo. Esa búsqueda de semejanza no necesita conocer características personales ni descripciones complejas del contenido: se apoya únicamente en el comportamiento observado, lo que lo convierte en un método especialmente útil cuando no se dispone de información adicional más allá del registro de interacciones.

Una vez identificado un grupo reducido de usuarios similares, la recomendación se basa en observar qué preguntas destacan en ese grupo y que el usuario actual todavía no ha realizado. A partir de aquí aparecen las dos variantes, que comparten estructura pero difieren en la intención.

En **preguntas que otros usuarios similares fallan**, se priorizan preguntas que suelen resultar problemáticas para usuarios con un comportamiento parecido. Conceptualmente, este enfoque funciona como un refuerzo preventivo: si a personas comparables les cuesta una pregunta, es probable que el usuario también necesite practicarla. Esto permite descubrir preguntas relevantes sin depender de que el usuario ya haya fallado previamente en ellas, y por tanto puede anticipar dificultades antes de que se repitan de forma persistente.

En **preguntas que otros usuarios similares aciertan**, la recomendación se apoya en el patrón contrario: se priorizan preguntas que usuarios similares suelen resolver correctamente. Esta variante puede entenderse como una forma de consolidación o progresión guiada por pares. Si usuarios comparables tienden a acertar ciertas preguntas, recomendarlas pue-

de servir para reforzar contenidos que encajan con el nivel actual del usuario o para mantener una experiencia menos frustrante en sesiones donde se busca estabilidad y confianza.

Como ocurre con cualquier método colaborativo basado en usuarios, estos algoritmos requieren una condición previa importante: debe existir un historial suficiente para poder comparar usuarios con sentido. Si el usuario ha respondido pocas preguntas, o si hay poco solapamiento de preguntas respondidas entre usuarios, resulta difícil identificar semejanzas fiables. Por ello, este tipo de recomendación suele ser más eficaz cuando el sistema ya ha acumulado interacciones y existe una base de usuarios con actividad suficiente. Además, conviene recordar que la recomendación por similitud tiende a reproducir los patrones de la comunidad; si el grupo de comparación es pequeño o poco diverso, la recomendación puede volverse limitada, insistiendo en preguntas muy marcadas por ese grupo.

4.1.7 Algoritmo basado en ítems: preguntas similares que fallan y preguntas similares que aciertan

Los algoritmos **basado en ítems: preguntas similares que fallan** y **basado en ítems: preguntas similares que aciertan** pertenecen a la familia del **filtrado colaborativo basado en ítems**. A diferencia del filtrado colaborativo basado en usuarios, aquí la comparación no se hace entre personas, sino entre preguntas. La intuición es que dos preguntas pueden considerarse similares si, en el uso agregado de la aplicación, tienden a comportarse de forma parecida dentro de los historiales de muchos usuarios.

La clave de este enfoque es que la similitud entre preguntas no se define por su contenido textual o por una etiqueta, sino por un patrón de uso. Si se observa que, para muchos usuarios, ciertas preguntas suelen ir de la mano en resultados, entonces existe una relación útil para recomendar. En este contexto aparecen dos formas de construir esa relación, que dan lugar a las dos variantes del algoritmo.

En **preguntas similares que fallan**, se entiende que dos preguntas están relacionadas si tienden a ser falladas conjuntamente por usuarios. Conceptualmente, esto permite detectar grupos de preguntas que comparten una dificultad similar o que exigen competencias próximas, aunque el sistema no conozca explícitamente esas competencias. La recomendación se genera tomando como punto de partida preguntas que el usuario ha fallado y buscando otras preguntas que, según el comportamiento global, suelen fallarse en los mismos perfiles. El resultado es un refuerzo dirigido no a repetir exactamente lo mismo, sino a expandir la práctica hacia preguntas cercanas a las dificultades ya observadas.

En **preguntas similares que aciertan**, la relación se construye en torno al acierto. Dos preguntas se consideran próximas si tienden a ser acertadas conjuntamente por usuarios. Desde un punto de vista conceptual, esto sugiere afinidad en el dominio o en el tipo de conocimiento requerido. La recomendación parte de preguntas que el usuario ya ha acertado y propone otras que suelen ir asociadas a ese mismo patrón de acierto, lo que se puede interpretar como una estrategia de consolidación y ampliación progresiva dentro de un nivel de dificultad parecido.

Una ventaja habitual del filtrado colaborativo basado en ítems es que tiende a ser más estable y reutilizable, ya que las relaciones entre preguntas pueden mantenerse razonablemente consistentes a medida que el número de usuarios crece. En lugar de depender de

encontrar usuarios muy parecidos para cada caso, el sistema se apoya en conexiones entre preguntas que se pueden explotar repetidamente. Sin embargo, también presenta limitaciones naturales. Para estimar similitudes fiables se necesita una base mínima de interacciones compartidas: si una pregunta es nueva o apenas se ha respondido, será difícil relacionarla con otras. Además, la calidad de la recomendación depende de que el comportamiento colectivo refleje patrones reales y no sea el resultado de coincidencias o de un uso todavía demasiado escaso.

En conjunto, estas dos variantes mantienen la misma estructura conceptual. La diferencia está en la señal que se utiliza para definir la relación entre preguntas, ya sea el fallo o el acierto, lo que permite orientar la sesión hacia refuerzo de dificultades o hacia consolidación de lo ya dominado.

4.2 Razonamiento de los algoritmos

Antes de describir el funcionamiento de cada algoritmo, conviene aclarar que todos siguen una estructura general muy parecida, ya que deben producir el mismo tipo de salida para la aplicación. En términos generales, cada ejecución parte de un conjunto de preguntas candidatas (que depende del criterio del algoritmo) y construye una sesión con un número fijo de preguntas. Para cada pregunta seleccionada, se generan además opciones de respuesta tipo test, asegurando que las alternativas incorrectas no coincidan con la respuesta correcta y que no se repitan entre sí. Finalmente, todas las preguntas se devuelven en un formato homogéneo que incluye los datos necesarios para presentar el audio, la respuesta correcta, las alternativas y la categoría.

Además, todos los algoritmos incorporan comprobaciones de consistencia para evitar resultados inválidos: por ejemplo, verificar que existen preguntas candidatas suficientes para construir una sesión y que el banco de preguntas permite generar opciones incorrectas distintas. Si alguna de estas condiciones no se cumple, o si ocurre un problema de acceso a datos, el resultado se devuelve indicando un estado de error junto con un mensaje explicativo.

4.2.1 Aleatorio simple

En este algoritmo, el conjunto de preguntas candidatas no se restringe por ningún criterio adicional: se considera todo el banco de preguntas como igualmente válido. La selección de las preguntas principales se realiza por puro azar, sin utilizar historial del usuario, rendimiento previo, categorías concretas ni información de otros usuarios. Por tanto, cada elección es independiente y no existe una prioridad interna que favorezca unas preguntas frente a otras.

Una consecuencia directa de este enfoque es que el algoritmo no garantiza una distribución equilibrada por categorías ni una progresión planificada en dificultad o contenido, ya que su comportamiento depende únicamente del azar. Del mismo modo, al no imponer restricciones de unicidad en la selección, es posible que una misma pregunta aparezca repetida dentro de una misma sesión si el banco es reducido o si el muestreo aleatorio así lo determina. En resumen, su funcionamiento se caracteriza por la neutralidad del criterio: todas las preguntas tienen la misma probabilidad de ser elegidas como base de la sesión.

4.2.2 Preguntas que no has hecho

Este algoritmo introduce un criterio de selección centrado en el avance por el banco de preguntas. En lugar de considerar todo el catálogo como igualmente probable, primero determina qué preguntas todavía no han sido respondidas por el usuario y las toma como conjunto prioritario. Con ello, la sesión se orienta a presentar contenido nuevo para el usuario, reduciendo repeticiones mientras exista material sin explorar.

El proceso separa las preguntas en dos grupos. El primer grupo está formado por las preguntas no realizadas por el usuario. El segundo grupo, que actúa como respaldo, se construye ordenando las preguntas ya respondidas según la última vez que fueron contestadas, de manera que queden primero las más antiguas. La sesión se compone dando prioridad al primer grupo y, solo si no es suficiente para completar la sesión, se rellena con preguntas del segundo grupo empezando por las menos recientes.

Una particularidad relevante de este algoritmo es que, cuando se seleccionan preguntas del grupo de las más antiguas, se actualiza la fecha de última respuesta asociada a esas preguntas para el usuario. De esta forma, dejan de considerarse inmediatamente como “antiguas” en ejecuciones posteriores, evitando que el sistema tienda a recomendar siempre las mismas preguntas de repaso y promoviendo rotación en el contenido revisado.

En resumen, su funcionamiento alterna de forma natural entre dos fases. Mientras existan preguntas nuevas, la recomendación se centra en descubrimiento y cobertura del catálogo. Cuando el usuario ya ha realizado todo, el algoritmo pasa a un modo de repaso basado en antigüedad, priorizando aquello que lleva más tiempo sin practicarse.

4.2.3 Categoría concreta

En este algoritmo, el criterio diferencial es que la sesión se construye exclusivamente con preguntas pertenecientes a una categoría determinada. Para ello, primero se filtra el banco de preguntas utilizando el identificador de categoría proporcionado como entrada. Si no existen preguntas asociadas a esa categoría, el algoritmo no puede continuar y devuelve un resultado de error indicando que no hay contenido disponible para ese filtro.

Una vez obtenido el conjunto de preguntas de la categoría, la selección de preguntas principales se realiza de forma aleatoria, pero siempre dentro de ese subconjunto. Esto provoca que la sesión mantenga un foco temático claro, ya que todas las preguntas pertenecen al mismo bloque de contenido, aunque la composición concreta de la sesión varíe en cada ejecución por el componente aleatorio.

En la generación de opciones de respuesta, el algoritmo intenta priorizar la coherencia dentro de la categoría: en primer lugar busca alternativas incorrectas también dentro de la misma categoría, y solo si no hay suficientes opciones distintas recurre a preguntas del resto del banco para completar las alternativas. De esta manera se evita que la sesión quede bloqueada cuando una categoría tiene pocas preguntas o pocas respuestas distintas, manteniendo al mismo tiempo el intento de que las opciones pertenezcan, siempre que sea posible, al mismo ámbito temático.

En resumen, el comportamiento del algoritmo se caracteriza por dos decisiones: restringir la selección de preguntas a una categoría concreta y mantener variedad dentro de esa res-

tricción mediante selección aleatoria, procurando además que las opciones incorrectas sean coherentes con la categoría siempre que el tamaño del conjunto lo permita.

4.2.4 Categoría que peor se te da y categoría que mejor se te da

En estos dos algoritmos, el criterio no lo decide el usuario manualmente, sino el propio sistema a partir del rendimiento histórico. El primer paso consiste en agrupar las respuestas previas del usuario por categoría y calcular, para cada una, cuántas preguntas se han respondido y cuántas se han contestado correctamente. Con esa información se estima el nivel de acierto en cada categoría y se descartan aquellas en las que el usuario aún no ha acumulado un mínimo de intentos, con el objetivo de evitar decisiones basadas en muestras demasiado pequeñas.

A partir de ese resumen, la selección de categoría se realiza de forma distinta según la variante. En **categoría que peor se te da**, se elige la categoría con menor porcentaje de aciertos dentro de las categorías válidas, enfocando la sesión hacia el refuerzo de aquello que más cuesta. En **categoría que mejor se te da**, se elige la categoría con mayor porcentaje de aciertos, orientando la sesión hacia la consolidación de lo que el usuario domina mejor.

Una vez determinada la categoría objetivo, la sesión se restringe a las preguntas de esa categoría. Si la categoría seleccionada no dispone de preguntas suficientes para construir una pregunta tipo test con varias alternativas, el algoritmo finaliza con un error, ya que no puede garantizar la generación de opciones coherentes. Cuando sí existe contenido suficiente, se generan las preguntas principales por selección aleatoria dentro de la categoría.

En la construcción de alternativas incorrectas, el algoritmo intenta obtenerlas dentro de la propia categoría para mantener consistencia temática. Si en algún caso no se encuentran suficientes opciones distintas mediante selección directa de otras preguntas, se completa el conjunto de alternativas utilizando respuestas distintas disponibles en esa misma categoría, evitando duplicados y evitando incluir la respuesta correcta de la pregunta principal.

En resumen, ambos algoritmos comparten el mismo flujo: primero se identifica automáticamente la categoría objetivo a partir del rendimiento del usuario, y después se construye la sesión dentro de esa categoría. La diferencia entre ellos está únicamente en el sentido del criterio, ya sea reforzar la categoría con peor rendimiento o consolidar la categoría con mejor rendimiento.

4.2.5 Preguntas que tiendes a fallar y preguntas que tiendes a acertar

Estos dos algoritmos comparten la misma estructura general: primero construyen un conjunto de preguntas candidatas a partir del historial del usuario y, después, priorizan dentro de ese conjunto aquellas que representan mejor el criterio buscado. La diferencia entre ambas variantes está en la señal que se utiliza para definir la prioridad: el fallo repetido o el acierto repetido.

En **preguntas que tiendes a fallar**, el algoritmo analiza el histórico del usuario pregunta a pregunta y resume, para cada una, cuántas veces se ha intentado y cuántas veces se ha fallado. Con esa información calcula una medida de dificultad personal, basada en la proporción de fallos respecto al total de intentos, y conserva únicamente preguntas en las que exista al menos un fallo previo. A partir de ahí, el conjunto candidato se ordena priorizando

primero una mayor proporción de fallos, después una mayor cantidad de fallos acumulados y, en caso de empate, aquellas que llevan más tiempo sin aparecer. Además, para que la selección final no se limite a “coger las primeras”, se aplica una selección con ponderación: las preguntas con más proporción de fallo y más fallos acumulados tienen más probabilidad de ser escogidas, pero sin eliminar por completo la variabilidad entre ejecuciones.

En **preguntas que tiendes a acertar**, el proceso es análogo, pero invirtiendo la lógica. Se resume el historial por pregunta contando intentos y aciertos, se calcula una medida de dominio personal basada en la proporción de aciertos, y se filtran las preguntas que el usuario ha acertado al menos una vez. El orden de prioridad se construye favoreciendo primero una mayor proporción de aciertos, después un mayor número de aciertos acumulados y, como criterio adicional, la antigüedad de la última interacción. Igual que en la variante anterior, la selección final no se hace de manera estrictamente determinista, sino mediante ponderación, aumentando la probabilidad de elegir preguntas con mejor desempeño sin convertir la sesión en una repetición idéntica.

En ambos casos, el algoritmo limita el tamaño del conjunto candidato para trabajar con un número controlado de preguntas priorizadas y, una vez elegidas las que formarán parte de la sesión, adjunta métricas del historial relacionadas con cada pregunta (por ejemplo, proporción de fallos o aciertos, número total de intentos y fecha de la última interacción). Finalmente, también se calculan estadísticas agregadas de la sesión, como un promedio del rendimiento asociado a las preguntas seleccionadas, lo cual permite interpretar el “perfil” general de la sesión generada: más orientada a refuerzo cuando predominan tasas altas de fallo, o más orientada a consolidación cuando predominan tasas altas de acierto.

4.2.6 Preguntas que otros usuarios similares fallan y preguntas que otros usuarios similares aciertan

Estos dos algoritmos son la primera aproximación colaborativa del sistema y comparten el mismo esquema general: en lugar de mirar únicamente el historial individual, buscan primero un grupo reducido de usuarios cuyo patrón de respuestas se parezca al del usuario actual y, a partir de ese grupo, extraen preguntas candidatas que el usuario todavía no ha respondido. La diferencia entre ambas variantes aparece en el tipo de preguntas que se consideran relevantes dentro del grupo de usuarios similares, ya sea por tendencia al fallo o por tendencia al acierto.

El proceso comienza recopilando el historial de respuestas del usuario y comprobando que exista una cantidad mínima de información para poder comparar. Con ese historial se identifica el conjunto de preguntas en las que el usuario ha interactuado, y se utiliza como base para localizar otros usuarios que hayan respondido también una parte significativa de esas mismas preguntas. A continuación, para cada candidato se calcula una medida de similitud basada en la coincidencia del patrón de aciertos y fallos en las preguntas que ambos han contestado. Solo se conservan usuarios cuya similitud sea positiva y, entre ellos, se selecciona un máximo de los más parecidos para formar el vecindario de referencia.

Una vez construido ese vecindario, el algoritmo genera recomendaciones buscando preguntas que cumplan dos condiciones: que hayan sido respondidas por usuarios similares y que el usuario actual aún no haya contestado. Este filtro es importante porque fuerza a que la

recomendación aporte contenido nuevo y evita sugerir preguntas ya vistas por el propio usuario, de modo que el componente colaborativo no se convierta en una repetición del historial personal.

A partir de aquí se bifurca el criterio según la variante. En **preguntas que otros usuarios similares fallan**, se seleccionan como candidatas las preguntas que, dentro del vecindario, muestran un patrón de dificultad: se consideran aquellas con un número mínimo de intentos en el grupo y con una proporción de fallos suficientemente alta. Las candidatas se priorizan dando preferencia a las que presentan mayor tendencia al fallo y, en segundo término, a las que acumulan más fallos totales. El resultado es una lista de preguntas que, para usuarios con comportamiento parecido, tienden a ser problemáticas y, por tanto, son plausibles puntos de refuerzo para el usuario actual.

En **preguntas que otros usuarios similares aciertan**, el criterio es simétrico. Se buscan preguntas que, dentro del vecindario, se resuelven con buen rendimiento: se exige también un mínimo de intentos y una proporción de aciertos suficientemente alta, y se prioriza primero por esa proporción y después por el número total de aciertos. Esta variante produce una lista de preguntas que el grupo similar suele resolver bien y que el usuario aún no ha realizado, lo que puede orientar la sesión hacia consolidación o progresión gradual en un nivel cercano al del usuario.

Un detalle relevante de implementación es que, una vez obtenida la lista priorizada de candidatas, la selección final de preguntas principales se toma directamente desde la cabecera de esa lista, hasta completar la sesión o agotar candidatas. Por ello, a diferencia de otros algoritmos donde se introduce aleatoriedad en la elección de las preguntas principales, aquí la variabilidad entre ejecuciones depende sobre todo de cómo cambie el vecindario de usuarios similares y de cómo evolucionen las estadísticas agregadas del grupo conforme se acumulan nuevas respuestas.

Cálculo de similitud entre usuarios

El paso clave en ambos algoritmos colaborativos descritos es la identificación de usuarios con patrones de respuesta similares al del usuario objetivo. Para cuantificar esta similitud de forma rigurosa, se utiliza una métrica matemática que compara cómo dos usuarios aciertan y fallan en las mismas preguntas.

Cada usuario se representa mediante un vector de respuestas binarias sobre el conjunto de preguntas que ha respondido. Para una pregunta p, el valor asociado es:

$$r_{u,p} = \begin{cases} 1 & \text{si la respuesta fue correcta} \\ 0 & \text{si la respuesta fue incorrecta} \end{cases}$$

Dado un usuario objetivo u y otro usuario v, la comparación se realiza únicamente sobre el conjunto de preguntas comunes $P_{u,v}$ que ambos han respondido. Si el número de preguntas comunes es insuficiente, la similitud no se considera fiable y se descarta la comparación.

Coeficiente de correlación de Pearson

La similitud entre dos usuarios u y v se calcula mediante el coeficiente de correlación de Pearson, definido como:

$$\text{sim}(u,v) = \frac{\sum_{p \in P_{u,v}} (r_{u,p} - \bar{r}_u)(r_{v,p} - \bar{r}_v)}{\sqrt{\sum_{p \in P_{u,v}} (r_{u,p} - \bar{r}_u)^2} \sqrt{\sum_{p \in P_{u,v}} (r_{v,p} - \bar{r}_v)^2}}$$

donde $\bar{r}_u$ y $\bar{r}_v$ representan la media de respuestas del usuario u y del usuario v respectivamente, calculadas únicamente sobre las preguntas comunes.

El valor de esta métrica se encuentra en el intervalo $[-1, 1]$. En la implementación realizada, solo se consideran usuarios con similitud estrictamente positiva, ya que valores negativos indican patrones de respuesta opuestos y no aportan información útil para la recomendación.

Relación con la implementación

En el sistema implementado, el cálculo de la similitud se realiza tras verificar que ambos usuarios han respondido un número mínimo de preguntas en común. A continuación, se calcula la media de respuestas de cada usuario y se aplica directamente la fórmula de Pearson sobre los valores binarios de acierto y fallo.

Una vez obtenidas las similitudes, los usuarios se ordenan de forma descendente y se selecciona un número máximo de los más similares para formar el vecindario de referencia. Este vecindario es el que se utiliza posteriormente para calcular las tasas de acierto o fallo por pregunta, que determinan la selección final de preguntas recomendadas en ambas variantes del algoritmo.

4.2.7 Algoritmo basado en ítems: preguntas similares que fallan y preguntas similares que aciertan

Estos dos algoritmos trasladan la idea colaborativa desde los usuarios hacia las **preguntas**. En lugar de buscar personas parecidas al usuario, lo que hacen es preguntarse qué preguntas se parecen entre sí según el comportamiento histórico de muchos estudiantes. La intuición es sencilla: si, en el uso real, ciertos usuarios tienden a fallar (o acertar) dos preguntas de forma conjunta, entonces esas preguntas comparten algún tipo de relación práctica, ya sea porque exigen una habilidad similar, porque pertenecen a un mismo tipo de sonido o porque provocan confusiones parecidas. Esa relación se convierte en una pista para recomendar contenido nuevo.

El funcionamiento se apoya en dos fases claramente separadas. En primer lugar, se construye un **punto de partida** a partir del propio historial del usuario. En la variante de **preguntas similares que fallan**, el sistema toma como base las preguntas que el usuario ha fallado al menos una vez. En la variante de **preguntas similares que aciertan**, la base son las preguntas que el usuario ha acertado. En ambos casos se exige un mínimo de preguntas base para evitar recomendaciones poco fiables; si el usuario solo tiene una interacción aislada, no existe suficiente contexto para hablar de similitud.

La segunda fase consiste en **expandir** esa base hacia preguntas nuevas. Para ello, el algoritmo descarta de antemano todas las preguntas que el usuario ya haya respondido, de modo que las recomendaciones aporten contenido inédito. Con el resto de preguntas disponibles se evalúa, una por una, si se parecen a alguna de las preguntas base. Esta similitud no se decide por metadatos (como categoría o dificultad), sino por comportamiento

agregado: se buscan usuarios que hayan respondido ambas preguntas y se analiza si, entre esos usuarios, los resultados en una y otra tienden a moverse en la misma dirección.

En la variante centrada en **fallos**, la similitud se calcula fijándose específicamente en el hecho de fallar. Dicho de otra forma, se estudia si dos preguntas tienden a ser falladas por los mismos usuarios. Si esa relación es consistente y, además, es positiva (es decir, si ambas preguntas se comportan de forma parecida), se considera que existe una conexión útil. En la variante centrada en **aciertos** se mantiene el mismo mecanismo, pero observando el patrón de acierto: se busca si dos preguntas tienden a ser acertadas por los mismos usuarios. En ambos casos, si hay muy pocos usuarios en común entre dos preguntas, la comparación se descarta para evitar que una coincidencia puntual se interprete como una relación real.

Una vez estimada la similitud entre la pregunta candidata y cada una de las preguntas base, el algoritmo se queda con un único valor representativo: la **máxima similitud** observada. Esto tiene un efecto práctico importante: una pregunta nueva se considera buena candidata si se parece mucho, al menos, a una de las preguntas que el usuario ha fallado (o acertado). Con ese criterio se filtran candidatas mediante un umbral mínimo, se ordenan de mayor a menor similitud y se seleccionan las primeras para formar la sesión.

Como resultado, **preguntas similares que fallan** tiende a proponer preguntas que, por patrones globales de uso, suelen estar asociadas a dificultades parecidas a las que ya ha mostrado el usuario. Por su parte, **preguntas similares que aciertan** tiende a recomendar preguntas que encajan con un perfil de dominio ya observado, funcionando como una extensión natural de lo que el usuario suele resolver bien. En ambos algoritmos, la recomendación final se apoya en evidencia agregada de la comunidad, pero aterrizada en el historial concreto del usuario mediante la selección de preguntas base.

Formalización del cálculo de similitud entre preguntas

Sea U el conjunto de usuarios y Q el conjunto de preguntas del sistema. Para cada usuario $u \in U$ y pregunta $q \in Q$, se define la variable binaria:

$$r_{u,q} = \begin{cases} 1 & \text{si el usuario responde correctamente la pregunta} \\ 0 & \text{si el usuario falla la pregunta} \end{cases}$$

Dadas dos preguntas q_i y q_j, se considera únicamente el conjunto de usuarios que han respondido ambas preguntas:

$$U_{ij} = \{u \in U \mid r_{u,q_i} \text{ y } r_{u,q_j} \text{ están definidos}\}$$

Si $|U_{ij}| < 3$, la similitud entre ambas preguntas se fija a cero para evitar inferencias basadas en muestras insuficientes.

Similitud basada en aciertos En el algoritmo de **preguntas similares que aciertan**, la similitud entre dos preguntas se calcula mediante la correlación de Pearson aplicada directamente sobre los valores de acierto:

$$\text{sim}(q_i, q_j) = \frac{\sum_{u \in U_{ij}} (r_{u,q_i} - \bar{r}_{q_i})(r_{u,q_j} - \bar{r}_{q_j})}{\sqrt{\sum_{u \in U_{ij}} (r_{u,q_i} - \bar{r}_{q_i})^2 \sum_{u \in U_{ij}} (r_{u,q_j} - \bar{r}_{q_j})^2}}$$

donde $\bar{r}_{q_i}$ y $\bar{r}_{q_j}$ representan el porcentaje medio de aciertos de cada pregunta entre los usuarios comunes. Únicamente se conservan valores positivos de correlación, fijando la similitud a cero en caso contrario.

Similitud basada en fallos En el algoritmo de **preguntas similares que fallan**, se invierte la variable de respuesta para centrarse explícitamente en el fallo:

$$f_{u,q} = 1 - r_{u,q}$$

La similitud se calcula de forma análoga, aplicando la correlación de Pearson sobre los vectores de fallos (f_{u,q_i}, f_{u,q_j}), manteniendo de nuevo únicamente correlaciones positivas.

Estrategia de selección de preguntas recomendadas

Sea B_u el conjunto de preguntas base del usuario u, definido como:

$$B_u = \begin{cases} \{q \mid q \text{ ha sido fallada por } u\} & \text{(algoritmo negativo)} \\ \{q \mid q \text{ ha sido acertada por } u\} & \text{(algoritmo positivo)} \end{cases}$$

Para cada pregunta candidata q_c no respondida previamente por el usuario, se calcula su similitud con cada pregunta base y se conserva el valor máximo:

$$\text{score}(q_c) = \max_{q_b \in B_u} \text{sim}(q_b, q_c)$$

Solo se consideran preguntas cuyo valor supere un umbral mínimo de similitud ($\text{score}(q_c) > 0.1$). Finalmente, las preguntas candidatas se ordenan de forma descendente según su puntuación y se seleccionan las mejores para conformar la sesión de entrenamiento.

Este enfoque garantiza que cada recomendación esté justificada por una relación fuerte con, al menos, una de las preguntas que caracterizan el comportamiento previo del usuario, manteniendo un equilibrio entre personalización y evidencia agregada.

4.3 JUSTIFICACIÓN DE LAS TECNOLOGÍAS UTILIZADAS

En este Complemento de Trabajo de Fin de Grado se han empleado tecnologías orientadas a la implementación de algoritmos y a su exposición como servicio dentro de una arquitectura basada en microservicios. El objetivo del complemento es exclusivamente ejecutar distintos algoritmos de recomendación para proponer preguntas, a partir de la información disponible en la base de datos.

Para la implementación de los algoritmos se ha utilizado **Python**. La elección de Python se fundamenta en su amplia adopción dentro de la comunidad de desarrollo e investigación,

especialmente en aplicaciones que combinan lógica algorítmica con aprendizaje automático y análisis de datos. La literatura destaca su sintaxis clara, su bajo coste de desarrollo y su capacidad para facilitar ciclos de experimentación rápidos, lo que resulta especialmente adecuado en contextos donde se comparan múltiples enfoques algorítmicos [11]. Además, Python se integra de forma natural con entornos web y arquitecturas basadas en microservicios, lo que lo convierte en una opción habitual para servicios que combinan lógica de recomendación y exposición mediante APIs.

Para publicar el recomendador como un microservicio se ha utilizado **Flask**. Flask es un microframework web para Python caracterizado por su enfoque minimalista y flexible, que permite construir APIs REST de forma sencilla y eficiente. Este tipo de frameworks ligeros resulta especialmente adecuado para microservicios con responsabilidades bien definidas, donde el servicio se limita a recibir peticiones HTTP, ejecutar un algoritmo y devolver resultados, sin necesidad de incorporar componentes adicionales que incrementen la complejidad del sistema [12].

Por último, se ha empleado **Docker** para el despliegue y el aislamiento del microservicio. Docker permite empaquetar una aplicación junto con todas sus dependencias en contenedores portables, garantizando un comportamiento consistente entre distintos entornos de ejecución. En arquitecturas de microservicios, la literatura identifica la contenerización como un elemento clave para mejorar la independencia entre servicios, facilitar el despliegue y reducir el impacto de incidencias en otros componentes del sistema [13, 14]. Además, el uso de contenedores favorece la mantenibilidad y escalabilidad del sistema, permitiendo desplegar o reiniciar servicios de forma independiente [15].

En conjunto, el uso combinado de Python, Flask y Docker permite construir un microservicio de recomendación modular, fácilmente desplegable y mantenible, alineado con prácticas modernas de desarrollo de software basadas en microservicios, separación de responsabilidades y contenerización de aplicaciones.

5 Conclusión y trabajo futuro

5.1 Conclusiones

A lo largo de este Complemento de Trabajo de Fin de Grado se ha diseñado e implementado un sistema de recomendación orientado a personalizar las sesiones de práctica dentro de una aplicación de auscultación para estudiantes de medicina. El objetivo principal ha sido sustituir una selección fija o manual de preguntas por un mecanismo automatizado, capaz de utilizar la información generada durante el uso real de la aplicación para proponer, en cada sesión, preguntas que resulten coherentes con el estado de aprendizaje del usuario y con el contenido disponible en el banco.

En el plano técnico, el complemento ha cristalizado en un conjunto de algoritmos que cubren diferentes niveles de complejidad y distintos escenarios de disponibilidad de datos. Por un lado, se han incluido métodos base que permiten generar sesiones sin necesidad de historial previo, lo cual resulta esencial en situaciones de arranque en frío o cuando el sistema aún no dispone de interacciones suficientes. Por otro lado, se han incorporado algoritmos centrados en el rendimiento individual, que explotan el histórico de aciertos y fallos para reforzar debilidades concretas o consolidar aspectos que el usuario domina. Finalmente, se han integrado estrategias colaborativas tanto basadas en usuarios como basadas en preguntas, que aprovechan patrones agregados en la comunidad para recomendar contenido nuevo a partir de similitudes observadas en el comportamiento de otros estudiantes.

Desde una perspectiva funcional, el sistema resultante permite generar sesiones completas de práctica a partir de un repositorio de preguntas amplio y en crecimiento, manteniendo un criterio consistente y repetible. Además, el hecho de disponer de varios algoritmos aporta flexibilidad: el sistema no queda atado a una única forma de recomendar, sino que puede alternar entre exploración y refuerzo, entre variedad y repetición controlada, o entre personalización estrictamente individual y recomendaciones derivadas de patrones colectivos. Esta diversidad de enfoques no solo amplía la capacidad del sistema para adaptarse a perfiles de usuario distintos, sino que también facilita comparar comportamientos y evaluar qué estrategias resultan más útiles en un contexto de entrenamiento auditivo como el de la auscultación.

Como conclusión personal, este complemento me ha permitido profundizar en el diseño de sistemas de recomendación desde una perspectiva aplicada, entendiendo de forma práctica cómo influyen la calidad de los datos, el historial de interacción y la estructura del repositorio en la capacidad de personalizar la experiencia. Considero especialmente valioso haber implementado algoritmos de distinta naturaleza, ya que me ha dado una visión más completa sobre cuándo conviene utilizar estrategias simples y cuándo merece la pena recurrir a métodos colaborativos para aprovechar la información colectiva generada por los usuarios.

5.2 TRABAJO FUTURO

Este complemento, al igual que el proyecto principal, dista mucho de estar cerrado: el sistema de recomendación implementado ofrece una base funcional y variada, pero todavía existen muchas líneas de mejora que podrían explorarse para aumentar tanto la calidad de las recomendaciones como su utilidad real en un contexto de aprendizaje.

Una primera mejora importante sería incorporar un marco de evaluación más sólido para comparar algoritmos de forma objetiva. Hasta ahora se dispone de distintos enfoques de recomendación, pero en un futuro sería interesante medir de manera sistemática qué impacto tiene cada uno en el progreso del estudiante. No se trata únicamente de observar el porcentaje de aciertos, sino de estudiar si el alumno mejora con el tiempo en categorías concretas, si retiene lo aprendido tras periodos sin práctica o si el sistema logra mantener una exposición equilibrada a diferentes tipos de sonidos. Esto permitiría justificar con datos qué estrategias son más adecuadas según el tipo de usuario o el momento del aprendizaje.

Relacionado con lo anterior, también sería útil desarrollar estrategias híbridas. En la práctica, un sistema de recomendación rara vez se beneficia de utilizar un único criterio de forma exclusiva, ya que eso puede volver la experiencia repetitiva o demasiado sesgada hacia un mismo patrón. En el futuro se podrían combinar varios enfoques dentro de una misma sesión, por ejemplo mezclando preguntas nuevas con preguntas de refuerzo, o alternando recomendaciones individuales con recomendaciones colaborativas. El objetivo sería encontrar un equilibrio natural entre exploración, consolidación y corrección de errores.

Por último, conforme crezca el banco de preguntas y el número de usuarios activos, será relevante estudiar la escalabilidad del sistema de recomendación. Los algoritmos colaborativos pueden requerir cálculos más costosos a medida que aumenta el volumen de interacciones, por lo que sería interesante plantear optimizaciones y estrategias de precálculo que permitan mantener el rendimiento sin perder personalización. Todo ello contribuiría a consolidar el sistema de recomendación como un componente más sólido, útil y sostenible dentro de la aplicación de entrenamiento en auscultación.

BIBLIOGRAFÍA

[1] M. F. Aljunid *et al.*, "A collaborative filtering recommender systems: Survey," *Neurocomputing*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0925231224014899

[2] D. Jannach, L. Lerche, and M. Zanker, "Recommending based on implicit feedback," in *Social Information Access*, 2018. [Online]. Available: https://web-ainf.aau.at/pub/jannach/files/BookChapter_Social_Information_Access_2018.pdf

[3] Unknown, "Job recommendations: Benchmarking of collaborative filtering methods for classifieds," *Electronics*, 2025. [Online]. Available: https://www.mdpi.com/2079-9292/13/15/3049

[4] W. contributors, "Item-item collaborative filtering," 2025. [Online]. Available: https://en.wikipedia.org/wiki/Item-item_collaborative_filtering

[5] M. Mansoury *et al.*, "Feedback loop and bias amplification in recommender systems," *arXiv*, 2020. [Online]. Available: https://arxiv.org/abs/2007.13019

[6] B. S. Greg Linden and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *acm digital library*, vol. 7, January 2023. [Online]. Available: https://dl.acm.org/doi/10.1109/MIC.2003.1167344

[7] Netflix, "How netflix's recommendations system works." [Online]. Available: https://help.netflix.com/en/node/100639

[8] ——, "Recommendations," *Netflix Research*. [Online]. Available: https://research.netflix.com/research-area/recommendations

[9] Spotify, "Five ways to make your discover weekly playlists even more personalized," May 2019. [Online]. Available: https://newsroom.spotify.com/2019-05-02/five-ways-to-make-your-discover-weekly-playlists-even-more-personalized/

[10] Tiktok, "How tiktok recommends content." [Online]. Available: https://support.tiktok.com/en/using-tiktok/exploring-videos/how-tiktok-recommends-content

[11] K. Mani and A. K. Shenoy, "Machine learning models in web applications: A comprehensive review," *ICT Express*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405959525001304

[12] V. K. Yepuri, V. K. Polamarasetty, S. Donthi, and A. K. R. Gondi, "Containerization of a polyglot microservice application using docker and kubernetes," *arXiv:2305.00600*, 2023. [Online]. Available: https://arxiv.org/abs/2305.00600

[13] A. L. Mhatre, “Microservices architecture using docker and kubernetes,” *International Journal for Multidisciplinary Research (IJFMR)*, 2023. [Online]. Available: https://www.researchgate.net/publication/387762754_Microservices_Architecture_Using_Docker_and_Kubernetes

[14] L. Baresi, G. Quattrocchi, and D. A. Tamburri, “Microservice architecture practices and experience: a focused look on docker configuration files,” *arXiv:2212.03107*, 2022. [Online]. Available: https://arxiv.org/abs/2212.03107

[15] H. U. Gaonkar, “Containerization (docker) and orchestration (kubernetes) in microservices,” *International Research Journal of Modernization in Engineering Technology and Science*, 2025. [Online]. Available: https://www.irjmets.com/upload_newfiles/irjmets70700006622/paper_file/irjmets70700006622.pdf

UNIVERSIDAD DE ALMERÍA



Resumen/Abstract

Este Complemento de Trabajo de fin de grado implementa un sistema de recomendación para una app que entrena a estudiantes de medicina a reconocer sonidos corporales con audios reales. Incluye 11 algoritmos (básicos, basados en rendimiento e interacciones colaborativas por usuarios e ítems) para generar sesiones de práctica personalizadas según historial de aciertos/errores, dominio por categorías y patrones de similitud.

This Bachelor's thesis add-on implements a recommendation system for an app that trains medical students to recognize body sounds from real audio recordings. It integrates 11 algorithms (basic, performance-based, and collaborative user-/item-based) to build personalized practice sessions using past correctness, category mastery, and similarity patterns across users and questions.

GRADO EN INGENIERÍA INFORMÁTICA

2025/2026